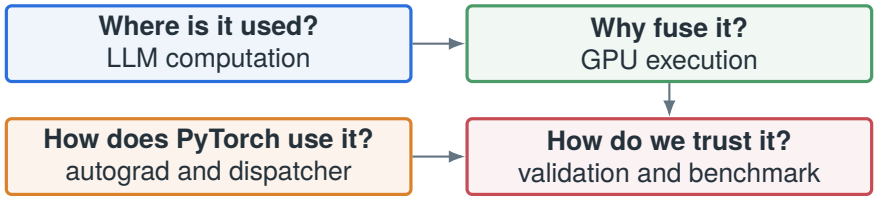
Fused Add + RMSNorm

Following one LLM operator through Triton and PyTorch

Forward kernel | backward rule | PyTorch registration | graph capture | benchmark

One operator, four questions



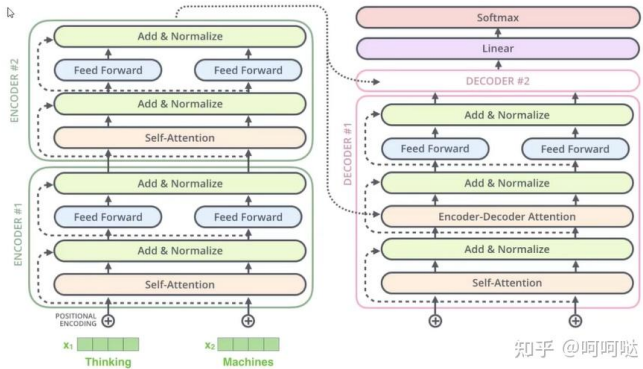
This lab follows one small but realistic computation through the complete operator lifecycle. The coding task is short because the main subject is the relationship between these four questions.

Key point: The two-line TODO is an entry point into operator development, not the whole lesson.

1. Position in an LLM

We first locate the operation in model computation and define what an operator means.

Transformer computation and the selected operation



What the image tells us

Transformer layers repeatedly combine a sublayer result with a residual path and then apply normalization.

The lab selects one adjacent pair:

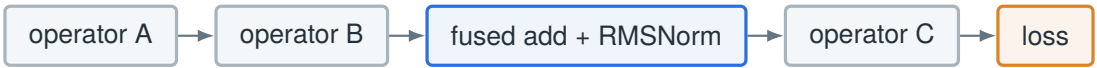
residual add + RMSNorm

This is a location map; exact LLM architectures vary.

Key point: We study a real model building block without running a complete LLM.

Operators and model execution

An **operator**, or **op**, is a named tensor computation with defined inputs and outputs. Matrix multiplication, softmax, and normalization are operators.



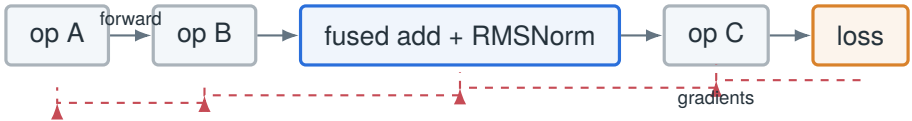
The logical operator in this lab is:

$$\text{fused_add_rms_norm}(x, \textit{residual}, \textit{weight}) \longrightarrow (\textit{out}, \textit{updated_residual})$$

Key point: The operator defines what is computed; its kernel defines how the GPU performs it.

Forward, backward, and autograd

The **forward pass** computes outputs from inputs. During training, the **backward pass** propagates gradients from the loss toward earlier values and parameters.



Autograd is PyTorch's system for organizing this reverse process. Each differentiable operator supplies a local gradient rule. Autograd connects the local rules into the backward pass of the complete model.

Key point: `grad_kernel.py` implements one local rule, not the backward pass of an entire LLM.

The operator contract

Inputs

$$x \in \mathbb{R}^{M \times N}$$

$$\mathit{residual} \in \mathbb{R}^{M \times N}$$

$$\mathit{weight} \in \mathbb{R}^{N}$$

The mean is calculated across the N columns independently for every row.

Forward computation

$$z = x + \mathit{residual}$$

$$\mathit{rrms} = \frac{1}{\sqrt{\mathrm{mean}(z^2) + \epsilon}}$$

$$\mathit{out} = z \cdot \mathit{rrms} \cdot \mathit{weight}$$

$$\text{return } (\mathit{out}, z)$$

There is one `rrms` value per row. The weight vector provides one learnable scale per column.

Key point: Returning the updated residual z also creates a gradient path.

2. From formula to fused kernel

We compare eager execution, fusion, and Triton before reading the forward code.

PyTorch eager execution

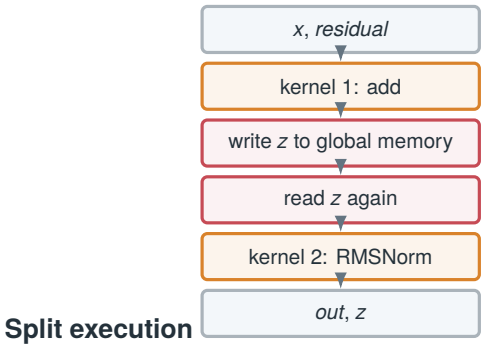
Eager mode is PyTorch's default behavior. Python reaches a tensor operation, PyTorch executes it immediately, and then Python moves to the next line.

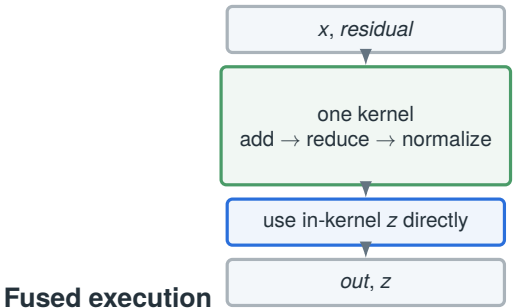
```
z = x + residual
variance = z.square().mean(dim=-1, keepdim=True)
rrms = torch.rsqrt(variance + eps)
out = z * rrms * weight
```

This code states the correct mathematics. It does not promise that the GPU will execute exactly one kernel. Individual operations normally dispatch to existing kernels, and intermediate tensors may pass through global memory.

Key point: One Python function and one GPU kernel are different concepts.

Why fusion can help



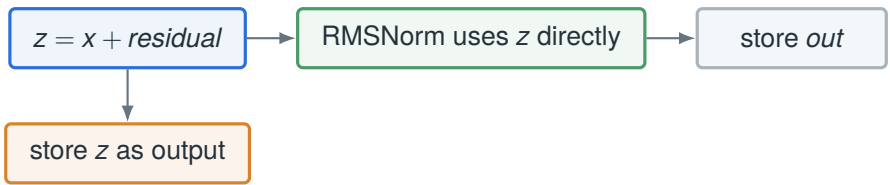


Key point: Fusion reduces a kernel launch and avoids reloading *z* for RMSNorm.

Why the updated residual is still stored

Fusion does not eliminate every store. The operator contract requires two outputs:

return $(out, updated_residual)$



The saved traffic is the second kernel's reload of z. The output store of z remains necessary.

Key point: The fused kernel stores z for the caller, not for a second RMSNorm kernel.

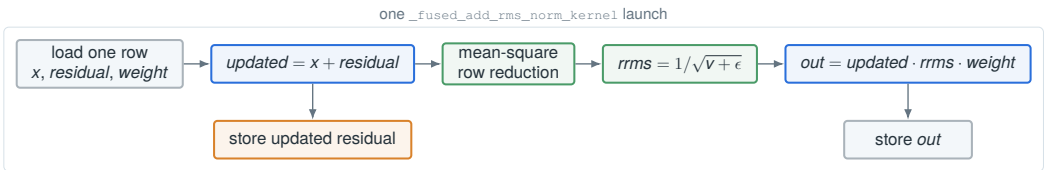
PyTorch, compilation, Triton, and CUDA

Level	What it provides	What the developer controls
PyTorch eager	Concise tensor and model code	Operations, not the exact final kernel boundary
`torch.compile`	Captures a computation graph and lets a backend optimize it	The graph; automatic fusion remains backend dependent
Triton	Python-like custom GPU kernel language	Program mapping, loads, reduction, stores, and kernel boundary
CUDA C++	Low-level GPU programming platform	Detailed hardware behavior with more code

Triton is not the only route to fusion. It is useful here because it makes the kernel boundary explicit with less code than a comparable CUDA implementation.

Key point: Triton gives explicit, readable kernel control; it is not the only route to fusion.

The fused forward kernel



One Triton program owns one row, so the same in-kernel `updated` value feeds both the returned residual and the RMSNorm calculation.

Key point: Add, reduction, normalization, and both output stores occur inside one kernel launch.

One Triton program per row



$$grid = (M,)$$

$$pid = \texttt{tl.program_id(0)}$$

Program `pid` processes row `pid`. A complete row is the natural unit because RMSNorm reduces across its N columns.
The row offset is:

$$pid \cdot N + cols$$

Key point: The parallel mapping follows the reduction dimension of the operator.

Block size, mask, and precision

Power-of-two block
If $N = 6$, Triton chooses `BLOCK_SIZE=8`:

$$\mathit{cols} = [0, 1, 2, 3, 4, 5, 6, 7]$$

$$\mathit{mask} = [T, T, T, T, T, T, F, F]$$

The mask disables positions outside the real row. Masked loads use zero, which does not change the reduction.

Float32 internal arithmetic
The kernel converts loaded values to `tl.float32` before addition and reduction. This improves numerical accuracy for bfloat16 inputs.
The output buffers use the input dtype, so the final stores convert values back to float32 or bfloat16 storage.

Key point: Block shape controls coverage; the mask controls validity; float32 controls reduction accuracy.

The two-line TODO in context

The kernel already computes the updated residual and the mean square:

```
updated = x + residual
tl.store(updated_residual_ptr + row_offsets, updated, mask=mask)

variance = tl.sum(updated * updated, axis=0) / n_cols
```

The TODO completes reciprocal RMS and output scaling:

```
rrms = 1.0 / tl.sqrt(variance + eps)
out = updated * rrms * weight
```

Key point: The final store is already present. These two lines complete an existing fused kernel; fusion comes from the whole kernel boundary.

Activity 1: forward implementation

Complete the region between `TODO BEGIN` and `TODO END` in `fused_rms_norm.py`, then run:

```
python3 fused_rms_norm.py
```

The script compares both outputs with the PyTorch reference for float32 and bfloat16 at several shapes.

Success signal

The script completes and reports small `out diff` and `res diff` values. For float32, the target is below 2×10^{-5}. This script intentionally prints no literal PASS line.

While it runs, identify the row dimension, the two outputs, and the internal arithmetic dtype.

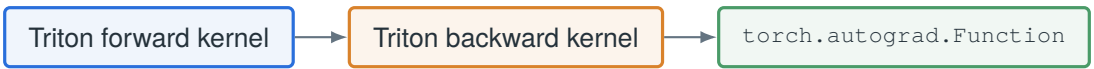
3. From kernel to PyTorch operator

A useful training operator needs gradients, registration, and graph-capture behavior.

Why a custom backward is needed

Ordinary PyTorch expressions are built from operations whose gradient rules autograd already knows.

A custom Triton kernel is opaque to autograd. PyTorch can launch it, but it cannot inspect arbitrary Triton code and automatically derive the gradient.

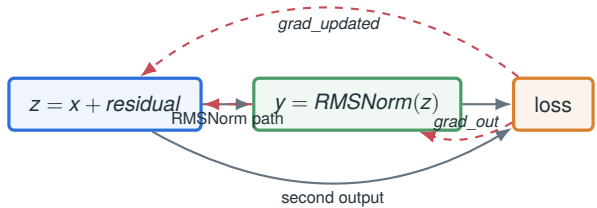


An inference-only operator may not need backward. This lab includes it so the operator can participate in training and so we can examine the full stack.

Key point: The backward kernel supplies the local gradient rule that autograd cannot infer from the Triton kernel.

Two gradient paths into the updated residual

The forward returns both $y = out$ and $z = updated_residual$. A loss may depend on both outputs.

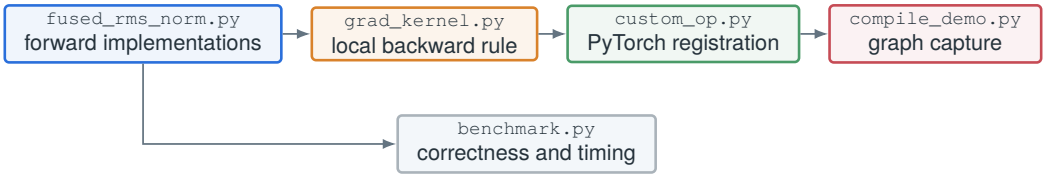


With $g = grad_out \cdot weight$ and $r = rrms$:

$$grad_z = gr - zr^3 \frac{\sum(gz)}{N} + grad_updated, \qquad grad_x = grad_{residual} = grad_z$$

Key point: Returning z creates a direct gradient path in addition to the RMSNorm path.

The operator lifecycle and repository structure



The files are not separate mini-projects. They answer successive questions about the same logical operator.

Key point: The repository layout mirrors the lifecycle from kernel to framework integration and measurement.

Important function names

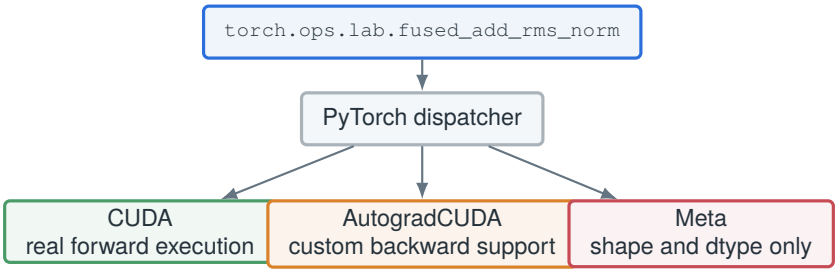
Function	Role
`_fused_add_rms_norm_kernel`	Actual one-kernel Triton forward implementation
`naive_add_rms_norm`	Readable PyTorch formulation; not used by validation
`torch_add_rms_norm`	Matched PyTorch reference used by forward, compile, and benchmark checks
`add_rms_norm`	Split Triton wrapper that launches add and RMSNorm separately
`fused_add_rms_norm`	Out-of-place wrapper that launches the fused kernel once
`fused_inplace`	Benchmark adapter that reuses input storage for outputs
`FusedAddRMSNorm`	Autograd wrapper around forward and backward kernels

Two naming traps matter: `add_rms_norm` is the split baseline, and `naive_add_rms_norm` does not guarantee one eager GPU kernel.

Key point: Use function behavior, not the shortest name, to identify the reference, split, and fused paths.

PyTorch dispatcher registration

The schema creates the stable name `torch.ops.lab.fused_add_rms_norm`. The dispatcher selects an implementation for the current execution context.

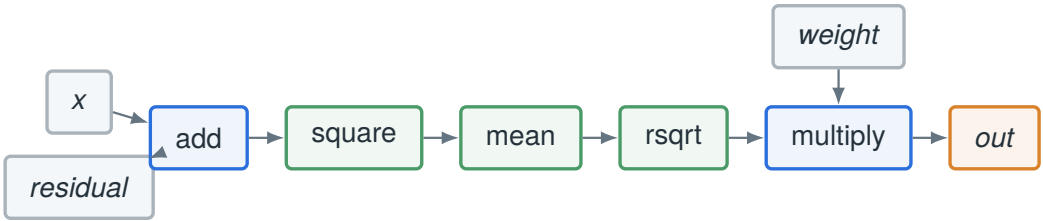


These are alternative paths, not three stages in sequence. The Meta path creates shape-and-dtype placeholders and launches no GPU calculation.

Key point: One logical operator can have context-specific implementations behind the same name.

Computation graph and graph capture

A **computation graph** represents tensor operations and their data dependencies. A compiler sees connected work instead of isolated Python lines.



`torch.compile` captures PyTorch work into a graph and sends it to a backend. A **graph break** occurs when capture cannot continue and execution must return to ordinary Python.

Key point: Graph capture gives the framework a connected view of the operator call and its dependencies.

What the compile demo proves

The lab runs:

```
python3 compile_demo.py --m 128 --n 256 --backend eager
```

Expected summary:

```
Dynamo graph summary: graphs=1, graph_breaks=0
PASS: torch.library operator runs eagerly and under torch.compile.
```

The graph summary describes capture of `module.forward`. The script also runs backward and validates gradients from eager and compiled-module outputs.

The `eager` backend checks capture and integration. It does not show Inductor code generation or prove a speedup.

Key point: The demo verifies graph compatibility and numerical correctness, not compiler performance.

Activity 2: backward and compile checks

Run the gradient validation:

```
python3 grad_kernel.py --m 128 --n 256 --dtype float32
```

It compares both outputs and the gradients of `x`, `residual`, and `weight` with PyTorch autograd. Then run the compile demonstration:

```
python3 compile_demo.py --m 128 --n 256 --backend eager
```

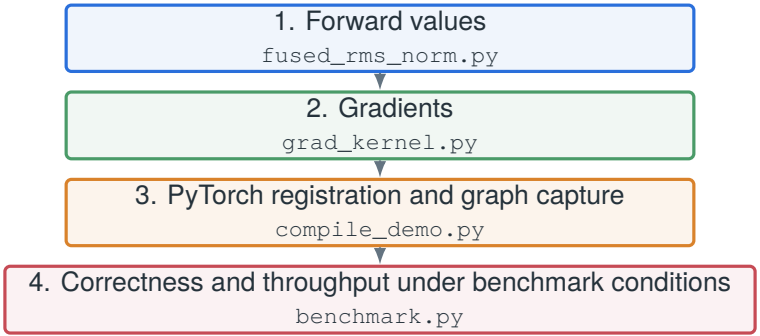
Success signals

Gradient PASS, then `graphs=1`, `graph_breaks=0`, and the final compile PASS.

4. Validation and performance

The final scripts test different properties: numerical correctness, integration, and throughput.

A validation ladder



A fast kernel can still be wrong. A correct raw kernel can still lack gradients or graph-capture support. Each step isolates a different property.

What the benchmark compares

Path	Purpose
PyTorch stepwise	High-level allocating context
Split Triton	Two-kernel context
Fused Triton	Student kernel with in-place benchmark adapter
FlagGems	Installed optimized in-place reference

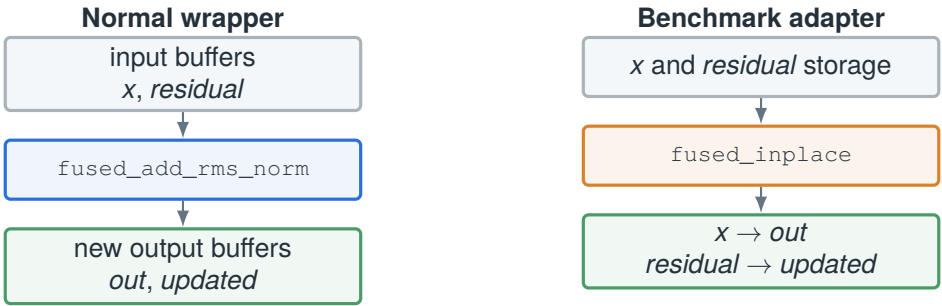
The main 90-percent target compares the two matched fused paths:

$$\frac{throughput_{fused}}{throughput_{FlagGems}} = \frac{time_{FlagGems}}{time_{fused}}$$

Input values are restored before every call. Restoration copies are excluded from the timed device interval.

Key point: The split and PyTorch paths provide context; the target applies to fused Triton versus FlagGems.

Out-of-place operator, in-place benchmark adapter



The adapter matches FlagGems storage behavior. It changes buffer use for measurement, not the mathematical operator contract.

Key point: Do not confuse the benchmark adapter with the normal user-facing wrapper.

Activity 3: performance comparison

Run both storage dtypes:

```
python3 benchmark.py --m 4096 --n 4096 --dtype float32
python3 benchmark.py --m 4096 --n 4096 --dtype bfloat16
```

Read four parts of the output:

1. latency in milliseconds;
2. estimated logical throughput in GB/s;
3. maximum absolute errors;
4. fused throughput as a percentage of FlagGems.

The final PASS requires numerical checks and at least 90 percent of FlagGems throughput under the benchmark's matched fused conditions.

Complete laboratory flow

```
python3 fused_rms_norm.py

python3 grad_kernel.py \
  --m 128 --n 256 --dtype float32

python3 compile_demo.py \
  --m 128 --n 256 --backend eager

python3 benchmark.py \
  --m 4096 --n 4096 --dtype float32

python3 benchmark.py \
  --m 4096 --n 4096 --dtype bfloat16
```

1. Complete the two-line TODO.
2. Check forward differences.
3. Check gradient PASS.
4. Check one graph, zero breaks, and compile PASS.
5. Interpret latency, errors, and the final benchmark result.

The complete operator story



- Fusion is a property of the generated kernel boundary, not the number of source lines.
- Triton gives explicit kernel control with less code than CUDA C++.
- Autograd composes operator-specific backward rules into model training.
- Validation separates numerical correctness, integration, and performance.

Key point: The lab follows one AI operator from its place in a model to a tested PyTorch implementation.

Questions and laboratory work

Keep the data flow in view: what is loaded, what is computed, what is returned, and which layer owns each behavior.